

Individual Contribution Reflection

Tracy Cui (540435227)

COMP4347 / COMP5347 Assignment 2 - MERN Quiz Game

Subsystem: Authentication and Security

1. Subsystem ownership. I owned the authentication and security slice: the User model, bcrypt password storage, JWT login, current-user lookup, public registration rules, role-based access checks, and login/register UI integration. My work connects the React auth forms to Express auth routes and then to MongoDB through the Mongoose User model.

2. Major technical challenge. The main technical challenge was preventing client-side role escalation while keeping admin access usable. The public register endpoint accepts user input, but it must never trust a submitted role. I resolved this by forcing registered accounts to role=user, moving password handling into User.setPassword, validating login/register payloads with Zod, and returning only safe user objects without passwordHash. I also hardened edge cases such as invalid credentials, expired or malformed tokens, non-admin access to admin routes, corrupted localStorage user data, and admin-mode login receiving a non-admin account.

3. Mermaid sequence diagram.

```
sequenceDiagram
    participant Browser as React Login/Register
    participant API as Express Auth Routes
    participant V as Zod Validators
    participant User as User Model
    participant JWT as JWT Middleware
    Browser->>API: POST /api/auth/register or /login
    API->>V: validate username/email/password
    API->>User: hash password or compare password
    User-->>API: safe user object, no passwordHash
    API-->>Browser: envelope with user and token
    Browser->>API: protected request with Bearer token
    API->>JWT: verify token and attach req.user
```

4. Review Mode design reflection. Review Mode still depends on authentication because history and review detail must be scoped to the current user. My design decision was to keep auth separate from quiz scoring while making it a reliable boundary around attempts: players can review only their own history, admins are blocked from player quiz routes, and invalid or expired tokens fail with clear envelope errors instead of leaking implementation detail.

5. Cross-system understanding. This subsystem depends on the full MERN stack: React collects input and displays feedback, Express exposes protected REST routes, middleware enforces role and validation rules, Mongoose persists the relevant state, and the shared response envelope keeps frontend error handling predictable.

6. Git commit history analysis. The table below cites meaningful commits from the GitHub Enterprise history and explains what each contributed technically.

Commit hashes: 67d5b87 e511754 c30c4c1 b57395f 763d25b 64f901e cdffc55 1642971 b85e9b6 dac2108 0801f4f c04d79f 0fd4b9e e0a6643 be281ed

Hash	Focus	Evidence
67d5b87	User model	Created User schema with role enum and timestamps.
e511754	Password security	Added bcrypt password hashing and compare helpers.
c30c4c1	Validation	Added Zod register/login validators and validate middleware.
b57395f	JWT middleware	Verified Bearer tokens and attached authenticated req.user.
763d25b	Register controller	Scaffolded registration handler and duplicate checks.
64f901e	Login controller	Issued JWT on valid credentials.
cdffc55	Current user	Added /me handler for protected user lookup.
1642971	Auth routes	Mounted auth routes with rate limiting and Swagger docs.
b85e9b6	Email support	Added email field and setPassword instance method.
dac2108	Register hardening	Required email, username pattern, and password 8-72 chars.
0801f4f	Role control	Forced public registration to role=user.
c04d79f	Error contract	Aligned login error message with API contract.
0fd4b9e	Register UI	Collected email in Register form and AuthContext.
e0a6643	Route UX	Supported adminMode and login redirects.
be281ed	Client guard	Handled corrupted user localStorage in ProtectedRoute.

7. Final reflection. My contribution was not only a list of commits; it was a bounded subsystem responsibility with evidence. The commits above show implementation, validation, integration, and final submission hardening. In the demo I can explain both my own files and how my subsystem interacts with authentication, quiz attempts, admin data, Review Mode, and MongoDB persistence.